

Portfolio Project Documentation

Pubmedqa Finetune

Deep-dive technical documentation — 6 sections



Generated April 07, 2026

Introduction

Readme

Deep Documentation — PubMedQA Fine-Tuning

Line-by-line explanations of every file, the concepts behind each decision, and interview-ready talking points.

Table of Contents

1. [Architecture](#) — How training and inference work end to end
2. [train.py](#) — Fine-tuning script explained line by line
3. [app.py](#) — Streamlit inference app explained line by line
4. [Concepts Glossary](#) — Every concept used in this project
5. [Interview Prep](#) — Questions and talking points

Section 1

Architecture

Architecture

The Big Picture

This project **fine-tunes** a pre-trained biomedical language model on a question answering task. Instead of building a model from scratch (which requires billions of examples), we take a model that already understands biomedical language and teach it one specific skill: reading a research abstract and deciding whether it answers a question with yes, no, or maybe.

There are two separate phases:



■■■ WARNING about uninitialized weights is expected and correct

■

▼

Step 5: Compute class weights

■

■■■ Count label distribution in training set (~55% yes, ~30% no, ~15% maybe)

■■■ `compute_class_weight("balanced")` → inverse frequency weights

■ yes: ~0.61, no: ~1.11, maybe: ~2.22

■■■ Higher weight = model penalized more for mistakes on that class

■

▼

Step 6: Train (3 epochs)

■

■■■ For each batch of 16 examples:

■ 1. Forward pass → logits

■ 2. `WeightedCrossEntropyLoss(logits, labels, weights)`

■ 3. Backward pass → gradients

■ 4. AdamW optimizer updates all 110M parameters

■■■ After each epoch: evaluate on validation set → log accuracy + macro F1

■■■ Save checkpoint if best macro F1 seen so far

■

▼

Step 7: Evaluate

■

■■■ Run inference on all 100 validation examples

■■■ Print per-class classification report (precision, recall, F1)

■

▼

Step 8: Push to HuggingFace Hub

■

■■■ Uploads: model weights, tokenizer, `config.json`

■■■ `config.json` contains `id2label` / `label2id` mappings

■■■ Model available at: huggingface.co/nikhilteja30/pubmedqa-bert

Inference Flow (app.py)

App startup (once per session)

■

```

■■■■ @st.cache_resource: load tokenizer + model from Hub
■■■■ First run: downloads ~440MB to ~/.cache/huggingface/
■■■■ Subsequent runs: loads from local cache instantly
■■■■ model.eval(): disables dropout for deterministic predictions
■■■■
▼
User submits question + abstract
■■■■
▼
predict() function
■■■■
■■■■ tokenizer(question, abstract, return_tensors="pt", ...)
■■■■   → {"input_ids": tensor(1,512), "attention_mask": tensor(1,512)}
■■■■
■■■■ torch.no_grad(): disables gradient tracking (inference only)
■■■■
■■■■ model(**inputs) → outputs.logits: tensor(1, 3)
■■■■   e.g. [2.1, -0.5, 0.3] (raw unnormalized scores)
■■■■
■■■■ F.softmax(logits, dim=-1) → [0.82, 0.08, 0.10]
■■■■   (probabilities that sum to 1.0)
■■■■
■■■■ argmax → 0 → "yes"
■■■■
▼
Display
■■■■
■■■■ Colored prediction label (:green[YES])
■■■■ Bar chart of confidence scores
■■■■ Probability table formatted as percentages

```

Why These Design Choices?

Choice	Why
BiomedBERT over general BERT	Pre-trained on PubMed — already understands medical vocabulary and sentence structure
pqa_labeled over pqa_unlabeled	1000 expert labels vs 61k auto-generated ones — quality over quantity for fine-tuning

Macro F1 as best-model metric	Class imbalance makes accuracy misleading — macro F1 treats all classes equally
Weighted cross-entropy loss	Without it, the model ignores the rare "maybe" class entirely
Push to Hub after training	Makes the app portable — anyone can run app.py without retraining
<code>@st.cache_resource</code> for model	Streamlit reruns the script on every interaction — without caching, the 440MB model would reload on every click

Section 2

Train

train.py — Line by Line

This script runs once on GPU (Colab) and does everything: loads data, tokenizes, trains the model, evaluates, and pushes to HuggingFace Hub.

Imports

```
import os
import numpy as np
from dotenv import load_dotenv
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
)
from sklearn.metrics import accuracy_score, f1_score, classification_report
```

- `os` — access environment variables (`os.environ.get("HF_TOKEN")`)
- `numpy` — array math for metric computation (`np.argmax`)
- `load_dotenv` — reads `.env` file and loads each line into `os.environ`
- `load_dataset` — downloads datasets from HuggingFace Hub with local caching
- `AutoTokenizer` — loads the right tokenizer for any model by name
- `AutoModelForSequenceClassification` — loads BiomedBERT + adds a classification head
- `TrainingArguments` — a config object holding all hyperparameters
- `Trainer` — the training loop (handles forward pass, backward pass, checkpointing, logging, Hub push)
- sklearn metrics — `accuracy_score`, `f1_score`, `classification_report`

Constants

```
MODEL_NAME = "microsoft/BiomedNLP-BiomedBERT-base-uncased-abstract"
```

The exact HuggingFace Hub model ID. Breaking it down:

- `microsoft` — published by Microsoft Research
- `BiomedNLP` — biomedical NLP group
- `BiomedBERT` — BERT pre-trained on biomedical text
- `base` — 12 transformer layers, 110M parameters (vs `large` which has 24 layers)
- `uncased` — text is lowercased before tokenization ("Statin" = "statin")
- `abstract` — pre-trained on PubMed **abstracts** only (not full papers)

```
DATASET_CONFIG = "pqa_labeled"
```

PubMedQA has three configs:

- `pqa_labeled` — 1000 examples, expert-annotated (what we use)
- `pqa_unlabeled` — 61,000 examples, no labels
- `pqa_artificial` — 211,000 examples, automatically labeled (lower quality)

```
LABEL2ID = {"yes": 0, "no": 1, "maybe": 2}
ID2LABEL = {0: "yes", 1: "no", 2: "maybe"}
```

Both mappings are stored in the model's `config.json` when pushed to Hub. This makes the model **self-describing** — anyone calling `pipeline("text-classification", model="nikhildeja30/pubmedqa-bert")` gets the correct labels automatically.

```
MAX_LENGTH = 512
```

BiomedBERT's hard maximum. Abstracts can be long — anything beyond 512 tokens is truncated from the right. In practice, ~95% of PubMedQA examples fit within 512 tokens.

`setup()`

```
def setup():
    load_dotenv()
    token = os.environ.get("HF_TOKEN")
    if not token:
        raise EnvironmentError("HF_TOKEN not found in .env file.")
    return token
```

`load_dotenv()` finds the `.env` file in the current directory and loads each `KEY=VALUE` line into `os.environ`. After this call, `os.environ["HF_TOKEN"]` works.

`.get()` is used instead of `os.environ["HF_TOKEN"]` because `.get()` returns `None` on a missing key instead of raising `KeyError`. The explicit check then gives a readable error message rather than a cryptic auth failure later.

`load_and_split()`

```
def load_and_split():
    dataset = load_dataset(DATASET_ID, DATASET_CONFIG)
    full = dataset["train"]
    split = full.train_test_split(test_size=0.1, seed=42)
    return split["train"], split["test"]
```

`load_dataset()` downloads from Hub on first call and caches to `~/.cache/huggingface/datasets/`. Subsequent calls use cache — no re-download.

`pqa_labeled` only has a "train" split — no pre-defined val/test. We create our own with `train_test_split(test_size=0.1, seed=42)`:

- `test_size=0.1` → 10% validation, 90% training → 100 val / 900 train
- `seed=42` → same random split every run (reproducibility)

Note: HuggingFace names the validation split "test" inside `train_test_split()`. This is just naming — it's our validation set.

`get_tokenize_fn()`

```
def get_tokenize_fn(tokenizer):
    def tokenize(examples):
        questions = examples["question"]
        contexts = [
            " ".join(item["contexts"])
            for item in examples["context"]
        ]
        tokenized = tokenizer(
            questions,
            contexts,
            max_length=MAX_LENGTH,
            truncation=True,
            padding="max_length",
        )
```

```

    tokenized["labels"] = [
        LABEL2ID[label] for label in examples["final_decision"]
    ]
    return tokenized
return tokenize

```

Why a closure? `dataset.map()` only accepts functions with the signature `fn(examples)`. But we also need the tokenizer. Returning a function from inside another function ("closure") is how we inject the extra variable — the inner `tokenize` function "remembers" the `tokenizer` from the outer scope.

Batched mode structure: When `batched=True`, `examples` is a dict of lists:

```

{
  "question": ["Q1", "Q2", ...],
  "context": [{"contexts": [...], ...}, {"contexts": [...], ...}, ...],
  "final_decision": ["yes", "no", ...]
}

```

So `examples["context"]` is a **list of dicts**, one per example. We iterate over it to access each example's context paragraphs.

Joining paragraphs: Each abstract in PubMedQA has 1-12 paragraphs stored as a list of strings. We join them with a space to create one continuous abstract string. The model receives the full abstract as one input.

Text pair tokenization: Passing two arguments to the tokenizer creates a text pair:

```

[CLS] question tokens [SEP] abstract tokens [SEP] [PAD] [PAD] ...
  ↑ token_type_id=0   ↑           ↑ token_type_id=1 ↑

```

The `[CLS]` token's final embedding is what flows into the classification head. The model uses `token_type_ids` to know which tokens are the question vs the abstract.

`padding="max_length"` pads all sequences to exactly 512 tokens with `[PAD]` tokens. The `attention_mask` is set to 0 for padding positions, so the model ignores them.

`truncation=True` cuts from the right side. If question + abstract > 512 tokens, the end of the abstract is cut off. This is acceptable because PubMedQA abstracts are structured — the key information is usually in the first paragraphs.

`compute_metrics()`

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    acc = accuracy_score(labels, preds)
    f1 = f1_score(labels, preds, average="macro")
    return {"accuracy": acc, "f1_macro": f1}
```

Called by Trainer after every evaluation epoch. `eval_pred` is a named tuple:

- `.predictions` — raw logits, shape `(100, 3)`
- `.label_ids` — true integer labels, shape `(100,)`

`np.argmax(logits, axis=-1)` picks the highest-scoring class for each example. `axis=-1` means "along the last dimension" (the 3 classes).

Why macro F1? With imbalanced classes (~55% yes, ~30% no, ~15% maybe), accuracy is misleading. A model that always predicts "yes" gets ~55% accuracy but is useless. Macro F1 computes F1 for each class separately and averages equally — it cares just as much about the rare "maybe" class as the common "yes" class.

The return dict key `"f1_macro"` matches `metric_for_best_model="f1_macro"` in `TrainingArguments` — that's how Trainer knows which metric to use for checkpoint selection.

Model Loading

```
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=3,
    id2label=ID2LABEL,
    label2id=LABEL2ID,
)
```

`AutoModelForSequenceClassification` detects from `config.json` that BiomedBERT is a BERT architecture and loads `BertForSequenceClassification`. This model is:

- **BiomedBERT body** — 12 transformer layers loaded from pre-trained weights
- **Classification head** — a single linear layer `(768 → 3)` that is **randomly initialized**

You will always see this warning:

```
Some weights of BertForSequenceClassification were not initialized from the model
checkpoint: ['classifier.bias', 'classifier.weight']
```

This is **correct and expected**. The BERT body weights are fine. Only the new classification head starts random and learns during training.

`id2label` and `label2id` get saved into `config.json` when the model is pushed to Hub, making it self-describing.

WeightedTrainer (Colab notebook version)

```
class WeightedTrainer(Trainer):
    def __init__(self, class_weights, **kwargs):
        super().__init__(**kwargs)
        self.class_weights = class_weights

    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.pop("labels")
        outputs = model(**inputs)
        loss_fn = nn.CrossEntropyLoss(weight=self.class_weights.to(outputs.logits.device))
        loss = loss_fn(outputs.logits, labels)
        return (loss, outputs) if return_outputs else loss
```

We inherit from `Trainer` and override only `compute_loss`. Everything else (training loop, checkpointing, evaluation, Hub push) stays unchanged.

Why `inputs.pop("labels")`? When we call `model(**inputs)` with labels present, the model computes its own loss internally. We don't want that — we want to compute our own weighted loss. Popping labels beforehand gives us raw logits without an internally computed loss.

`self.class_weights.to(outputs.logits.device)` — moves the weight tensor to the same device as the model (GPU/CPU). If we forgot `.to(device)`, PyTorch would raise an error about tensors on different devices.

Class weights computed with:

```
weights = compute_class_weight("balanced", classes=np.array([0, 1, 2]), y=train_labels)
```

"balanced" means $\text{weight} = \text{total_samples} / (\text{n_classes} \times \text{class_count})$. With ~55% yes, ~30% no, ~15% maybe, the weights come out roughly `[0.61, 1.11, 2.22]` — maybe gets ~3.6x more penalty than yes.

TrainingArguments

```

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    learning_rate=2e-5,
    warmup_steps=34,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="f1_macro",
    greater_is_better=True,
    fp16=True,
    push_to_hub=True,
    hub_model_id=HUB_MODEL_ID,
    hub_token=HF_TOKEN,
    report_to="none",
)

```

Argument	Value	Why
<code>num_train_epochs</code>	3	With 900 examples, more epochs cause overfitting
<code>per_device_train_batch_size</code>	16	GPU can handle 16; use 8 on CPU
<code>learning_rate</code>	2e-5	Lower than default (5e-5) — BiomedBERT is already domain-adapted, needs gentle updates
<code>warmup_steps</code>	34	~10% of total steps — ramps LR from 0 to 2e-5 to protect the random classification head from large early updates
<code>weight_decay</code>	0.01	L2 regularization — slightly penalizes large weights to reduce overfitting
<code>eval_strategy</code>	"epoch"	Evaluate after every epoch

<code>load_best_model_at_end</code>	True	If epoch 2 was best and epoch 3 overfit, reload epoch 2
<code>metric_for_best_model</code>	"f1_macro"	Best checkpoint = highest macro F1
<code>fp16</code>	True	Half-precision training on GPU — 2x faster, halves memory
<code>push_to_hub</code>	True	Auto-push checkpoints during training
<code>report_to</code>	"none"	Suppress wandb/tensorboard prompts

Training, Evaluation, Push

```
trainer.train()
```

Runs the full training loop. Trainer handles:

- Batching and shuffling
- Forward + backward pass every step
- AdamW optimizer with linear LR schedule
- Evaluation after each epoch
- Checkpoint saving

```
predictions = trainer.predict(val_ds)
preds = np.argmax(predictions.predictions, axis=-1)
print(classification_report(labels, preds, target_names=["yes", "no", "maybe"]))
```

`trainer.predict()` runs inference on the full val set without computing gradients.

`classification_report` shows per-class precision, recall, F1, and support — far more informative than a single number.

```
trainer.push_to_hub(commit_message="...")
```

Uploads the best checkpoint's weights + tokenizer + config to Hub. Creates the repo

`nikhildeja30/pubmedqa-bert` if it doesn't exist yet.

Section 3

App

app.py — Line by Line

The Streamlit inference app. Loads the fine-tuned model from HuggingFace Hub and provides a UI to test it.

Imports and Constants

```
import streamlit as st
import torch
import torch.nn.functional as F
import pandas as pd
from transformers import AutoTokenizer, AutoModelForSequenceClassification
```

- `streamlit` — the web app framework. Every `st.` call renders something in the browser.
- `torch` — PyTorch, used for the inference forward pass and tensor operations
- `torch.nn.functional as F` — functional API, used for `F.softmax()`
- `pandas` — creates the probability DataFrame for the table display
- `AutoTokenizer, AutoModelForSequenceClassification` — load from Hub by model ID

```
HUB_MODEL_ID = "nikhildeja30/pubmedqa-bert"
```

The app loads the model from Hub, not from a local file. This means the app is portable — anyone can clone and run it without retraining. The Hub acts as the model registry.

```
LABEL_COLORS = {"yes": "green", "no": "red", "maybe": "orange"}
```

Used with Streamlit's colored text syntax: `:green[**YES**]`. The color changes based on the prediction.

Model Loading

```
@st.cache_resource
def load_model():
    with st.spinner("Loading model from HuggingFace Hub (~440MB on first run)..."):
        tokenizer = AutoTokenizer.from_pretrained(HUB_MODEL_ID)
```

```
model = AutoModelForSequenceClassification.from_pretrained(HUB_MODEL_ID)
model.eval()
return tokenizer, model
```

Why `@st.cache_resource`?

Streamlit is not like a normal Python script. It **reruns the entire file** on every user interaction — every button click, every keystroke in a text box. Without caching, this would download and load the 440MB model on every single interaction, making the app unusable.

`@st.cache_resource` stores the return value (tokenizer + model) in memory after the first call. All subsequent reruns skip the function body and return the cached objects instantly.

Why `cache_resource` and not `cache_data`?

- `cache_data` — for serializable objects (DataFrames, lists, numbers). It pickles the return value.
- `cache_resource` — for non-serializable objects like ML models and database connections. It stores the object reference directly without pickling.

PyTorch models cannot be pickled reliably, so `cache_resource` is the correct choice.

`model.eval()` switches the model from training mode to evaluation mode. The key difference: it disables **dropout** layers. Dropout randomly zeros out neurons during training to prevent overfitting. During inference you want deterministic, consistent predictions — the same input should always produce the same output.

```
tokenizer, model = load_model()
```

This line is at module level — it runs once when the app starts. Subsequent reruns skip it via the cache.

`predict()`

```
def predict(question: str, abstract: str, tokenizer, model):
    inputs = tokenizer(
        question,
        abstract,
        return_tensors="pt",
        max_length=MAX_LENGTH,
        truncation=True,
        padding="max_length",
    )
```

Same tokenization as training — **this must match exactly**. If training used `max_length=512` and inference uses `max_length=256`, the model sees inputs in a format it was never trained on and produces garbage.

`return_tensors="pt"` returns PyTorch tensors instead of Python lists. The result is:

```
{
  "input_ids": tensor([[101, 2079, ..., 0, 0]]),      # shape: (1, 512)
  "attention_mask": tensor([[1, 1, ..., 0, 0]]),      # shape: (1, 512)
  "token_type_ids": tensor([[0, 0, ..., 0, 0]])      # shape: (1, 512)
}
```

Batch size is 1 (one example at a time).

```
with torch.no_grad():
    outputs = model(**inputs)
```

`torch.no_grad()` is a context manager that disables gradient tracking. During training, PyTorch tracks every operation to compute gradients (needed for backpropagation). During inference, gradients are never needed — disabling tracking saves significant memory and computation.

`model(**inputs)` unpacks the dict as keyword arguments:

```
model(input_ids=tensor(...), attention_mask=tensor(...), token_type_ids=tensor(...))
```

`outputs.logits` is the raw output — shape `(1, 3)`. Example: `[[2.1, -0.5, 0.3]]`

These are not probabilities — they're unnormalized scores. A logit of 2.1 doesn't mean 2.1% — it just means "yes" is the most likely class relative to the others.

```
probs = F.softmax(outputs.logits, dim=-1).squeeze().tolist()
```

`F.softmax(logits, dim=-1)` converts logits to probabilities that sum to 1. `dim=-1` applies the softmax along the last dimension (the 3 classes).

```
softmax([2.1, -0.5, 0.3]) → [0.818, 0.060, 0.122]
                        ↑ yes  ↑ no   ↑ maybe
```

`.squeeze()` removes the batch dimension: shape `(1, 3) → (3,)`

`.tolist()` converts the tensor to a Python list: `[0.818, 0.060, 0.122]`

```
pred_id = int(torch.argmax(torch.tensor(probs)).item())
return ID2LABEL[pred_id], probs
```

`argmax` finds the index of the highest probability. `.item()` converts a 0-dimensional tensor to a Python int. `ID2LABEL[0] → "yes"`.

UI Layout

```
st.set_page_config(page_title="PubMedQA Classifier", layout="wide")
```

Must be the **first Streamlit call** in the script. `layout="wide"` uses the full browser width instead of the narrow centered column default.

```
col1, col2 = st.columns(2)
with col1:
    question = st.text_area("Biomedical Research Question", height=120, ...)
with col2:
    abstract = st.text_area("Abstract / Context", height=300, ...)
```

`st.columns(2)` creates two equal-width columns side by side. The `with col1:` context manager directs all Streamlit calls inside it to render in that column. This gives a clean two-panel layout: question on the left, abstract on the right.

```
if st.button("Predict Answer", type="primary", use_container_width=True):
```

`st.button()` renders a button and returns `True` only on the rerun triggered by clicking it. The entire `if` block runs only when the button is clicked. `type="primary"` gives it the blue filled style. `use_container_width=True` stretches it full width.

```
if not question.strip() or not abstract.strip():
    st.warning("Please fill in both the question and the abstract context.")
else:
    with st.spinner("Running inference..."):
        label, probs = predict(question.strip(), abstract.strip(), tokenizer, model)
```

`st.spinner()` shows a loading animation while the block inside executes. `.strip()` removes leading/trailing whitespace so a space-only input counts as empty.

```
color = LABEL_COLORS[label]
st.markdown(f"## Prediction: :{color}[**{label.upper()}**]")
```

Streamlit's colored text syntax uses `:color[text]` inside markdown strings. The color changes based on the prediction — green for yes, red for no, orange for maybe.

```
prob_df = pd.DataFrame({
    "Answer": list(ID2LABEL.values()),
    "Confidence": probs,
})

col_chart, col_table = st.columns([2, 1])
with col_chart:
    st.bar_chart(prob_df.set_index("Answer"))
with col_table:
    st.dataframe(
        prob_df.style.format({"Confidence": "{:.1%}"})
        , hide_index=True
    )
```

`st.columns([2, 1])` creates unequal columns — the chart gets 2/3 of the width, the table gets 1/3.

`prob_df.set_index("Answer")` makes "yes"/"no"/"maybe" the x-axis labels for the bar chart.

`.style.format({"Confidence": "{:.1%}"})` formats the Confidence column as a percentage with one decimal place: `0.818` → `81.8%`.

`hide_index=True` removes the 0/1/2 row numbers from the table display.

Section 4

Concepts

Concepts Glossary

Every concept used in this project, explained in one place.

Transfer Learning

Taking a model trained on a large general task and adapting it to a specific smaller task.

Analogy: A doctor who spent 8 years in medical school already knows anatomy, physiology, and pharmacology. Teaching them to read cardiograms takes weeks, not years — the foundational knowledge transfers. Training a specialist from scratch with zero medical knowledge would take much longer.

In this project: BiomedBERT spent months pre-training on 21 million PubMed abstracts. It already knows medical vocabulary, how drug names relate to conditions, how research abstracts are structured. Fine-tuning on 900 PubMedQA examples takes ~8 minutes on GPU rather than weeks.

Fine-Tuning

Updating all (or some) weights of a pre-trained model on a new, smaller dataset for a specific task.

What actually happens: We add a new classification head (a linear layer with 3 outputs) on top of BiomedBERT and train the whole thing. The BERT body already has good weights — fine-tuning nudges them to be slightly better for our task. The new head starts random and learns from scratch.

Why not freeze the BERT body? For small datasets, freezing is common. For 900 examples of domain-specific text (PubMed abstracts) with a domain-specific base model (BiomedBERT), fine-tuning the whole model tends to work better — the body weights are already close to optimal for this domain.

Domain Adaptation

Pre-training a model on domain-specific data (medical, legal, financial) instead of general text.

Why it matters: General BERT (`bert-base-uncased`) was pre-trained on Wikipedia and books.

Medical text has completely different vocabulary patterns:

- "The patient presented with acute myocardial infarction" → general BERT struggles with this
- "MI patients treated with ACE inhibitors showed reduced LV dysfunction" → abbreviations, clinical terminology

BiomedBERT pre-trained on PubMed already understands these patterns. Using it instead of general BERT gives meaningfully better results on medical tasks without any extra work.

BERT Architecture

BERT (Bidirectional Encoder Representations from Transformers) is a transformer encoder that reads text in both directions simultaneously — unlike earlier models that read left-to-right only.

Key components:

- **Tokenizer** — splits text into sub-word pieces ("statin" → ["stat", "##in"]) and converts to integer IDs
- **Embeddings** — converts token IDs to 768-dimensional vectors
- **12 Transformer layers** — each layer refines the representation using self-attention
- **[CLS] token** — a special token prepended to every input; its final 768-dim output represents the whole sequence and flows into the classification head

Self-attention (what makes transformers powerful): every token looks at every other token and decides how much to "attend" to it. The word "it" in "The statin reduced inflammation because it blocked COX-2" can figure out "it" refers to "statin" by attending to the right context.

Tokenization and Text Pairs

Tokenization converts raw text into numbers the model understands.

Sub-word tokenization: Words are split into pieces. "uncharacteristically" might become ["un", "##character", "##istically"]. This handles rare and medical words gracefully — instead of having an unknown token, the model builds meaning from familiar sub-parts.

Text pair format: When two texts (question + abstract) are passed to BERT, the result is:

```
[CLS] question [SEP] abstract [SEP] [PAD] [PAD] ...  
  ↑           ↑           ↑  
special token separator token padding token
```

Three output arrays are produced:

- `input_ids` — integer ID for each token
- `attention_mask` — 1 for real tokens, 0 for padding (model ignores padding)
- `token_type_ids` — 0 for first segment (question), 1 for second (abstract)

The model uses `token_type_ids` to distinguish which part is the question and which is the context.

Classification Head

A linear layer added on top of BERT that converts the 768-dimensional `[CLS]` representation into class scores.

```
[CLS] final embedding (768-dim) → Linear(768, 3) → logits (3-dim)
```

This head is the only part of the model that starts with random weights. Everything else (BiomedBERT's 12 layers) is loaded from pre-trained weights. The head learns during fine-tuning from gradient updates.

Why [CLS]? BERT was designed so the [CLS] token attends to all other tokens through self-attention across all 12 layers. By the final layer, its representation has "seen" and aggregated information from the entire input — making it ideal for classification.

Logits, Softmax, and Probabilities

Logits are the raw unnormalized outputs from the classification head. Example: [2.1, -0.5, 0.3]

These are not probabilities — you can't interpret 2.1 as "21% chance of yes". They're just scores where higher means more likely.

Softmax converts logits to probabilities that sum to 1:

```
softmax([2.1, -0.5, 0.3]) → [0.82, 0.06, 0.12]
```

	yes	no	maybe
	82%	6%	12%

82% + 6% + 12% = 100%

ArgMax picks the class with the highest probability as the prediction:

```
argmax([0.82, 0.06, 0.12]) → 0 → "yes"
```

During training, **CrossEntropyLoss** applies softmax internally and computes how wrong the predicted probabilities are compared to the true label.

Class Imbalance

When the training data has significantly more examples of some classes than others.

In this project: PubMedQA `pqa_labeled` has ~55% yes, ~30% no, ~15% maybe. Only 1 in 7 examples is "maybe".

What happens without handling it: The model learns that predicting "yes" is rewarded more often and stops predicting "maybe" entirely. You end up with a biased model that achieves decent accuracy (55% by always saying yes) but is useless for the minority classes.

The fix — Weighted Cross-Entropy Loss: Each class gets a weight inversely proportional to its frequency. "Maybe" gets ~2.22x weight vs "yes" at 0.61x. Every time the model misclassifies a "maybe" example, the loss (and therefore the gradient update) is 3.6x larger. The model is forced to take minority classes seriously.

Macro vs Weighted F1

F1-score is the harmonic mean of precision and recall. It balances both — a model that's precise but misses everything (high precision, zero recall) gets a low F1.

Weighted F1 averages class F1s weighted by support (class frequency). Majority classes dominate.

- A model that always predicts "yes" gets decent weighted F1 because yes is 55% of examples.

Macro F1 averages class F1s equally regardless of frequency. Every class has equal say.

- That same model gets terrible macro F1 because no and maybe F1s are both 0.

For imbalanced problems, **macro F1 is the honest metric**. It's what we use as `metric_for_best_model` and what we track in the classification report.

AdamW Optimizer

The optimizer used by Trainer. An extension of Adam with **weight decay decoupled** from the gradient updates.

- **Adam** — adapts the learning rate per parameter based on gradient history. Parameters that receive rare gradients get larger updates.
- **Weight decay** — L2 regularization that shrinks weights toward zero each step. Prevents overfitting by penalizing large weights.
- **"W" = decoupled weight decay** — in standard Adam, weight decay interacts with the adaptive learning rates in a buggy way. AdamW fixes this by applying weight decay directly to the weights, separately from the gradient update.

In practice: `weight_decay=0.01` means weights shrink by 1% each step before the gradient update.

Learning Rate Warmup

Starting with a very small learning rate and gradually increasing it to the target over the first N steps.

Why? At the start of training, the classification head has random weights. A full-size learning rate update on random weights can cause very large, destructive gradient updates that damage the carefully pre-trained BiomedBERT weights. Warmup over 34 steps lets the head learn gradually without destabilizing the base model.

Step:	0	10	20	34	50	...
LR:	0.0	0.6e-5	1.2e-5	2e-5	2e-5	...
	↑ warmup phase ↑		↑ steady ↑			

HuggingFace Hub

A model hosting platform — think "GitHub for ML models."

What gets uploaded when you push:

- `model.safetensors` — the fine-tuned weights (~440MB for BiomedBERT)
- `config.json` — model architecture + `id2label/label2id` mappings
- `tokenizer_config.json`, `vocab.txt` — tokenizer files
- A model card (auto-generated README)

Why this matters: After pushing, anyone can use the model with a single line:

```
from transformers import pipeline
classifier = pipeline("text-classification", model="nikhildeja30/pubmedqa-bert")
```

No local weights needed, no label mapping needed — it's all in the Hub repo.

`@st.cache_resource`

A Streamlit decorator that caches the return value of a function across reruns.

Why it's needed: Streamlit reruns the entire Python script on every user interaction. Without caching, the 440MB model would be re-downloaded and re-loaded every time the user clicks a button or types in a text box.

`cache_resource` vs `cache_data`:

- `cache_data` — for data (DataFrames, lists, dicts). Pickles the return value for storage.
- `cache_resource` — for resources (models, DB connections). Stores the object reference directly.

ML models cannot be pickled reliably (they contain GPU tensors, file handles, etc.), so `cache_resource` is always the right choice for HuggingFace models in Streamlit apps.

`torch.no_grad()`

A context manager that disables PyTorch's automatic differentiation engine during inference.

During training, every tensor operation is recorded in a computational graph so gradients can be computed backward through it (backpropagation). This recording has memory and compute cost.

During inference, you never need gradients — you just want the output. `torch.no_grad()` skips the recording entirely:

- Significantly less memory usage
- Faster forward pass
- No risk of accidentally accumulating gradients

Always use `torch.no_grad()` during inference. Forgetting it doesn't break correctness but wastes resources.

Section 5

Interview Prep

Interview Prep

Questions you should be able to answer confidently about this project.

"Walk me through what this project does."

Strong answer:

"I fine-tuned Microsoft's BiomedBERT — a BERT model pre-trained on 21 million PubMed abstracts — on the PubMedQA dataset. Given a biomedical research question and an abstract, the model classifies the answer as yes, no, or maybe. The key challenge was class imbalance: 'maybe' made up only 15% of examples, so without intervention the model completely ignored it. I solved this with weighted cross-entropy loss, which boosted the 'maybe' F1 from 0.00 to 0.37 and overall macro F1 from 0.36 to 0.51. The fine-tuned model is deployed on HuggingFace Hub and served through a Streamlit app."

"Why did you choose BiomedBERT over BERT?"

Strong answer:

"Domain adaptation. General BERT was pre-trained on Wikipedia and books — it's never seen 'myocardial infarction' or 'ACE inhibitor' in meaningful context. BiomedBERT was specifically pre-trained on 21 million PubMed abstracts, so it already understands biomedical vocabulary, abbreviations, and how medical sentences are structured. Using a domain-specific base model gives meaningfully better results on medical NLP tasks without any extra work."

"What is fine-tuning? How is it different from training from scratch?"

Strong answer:

"Training from scratch means randomly initializing all weights and learning everything from your data. Fine-tuning starts with a pre-trained model and updates its weights on a smaller, task-specific dataset. In this project, BiomedBERT's 110 million parameters were already tuned to understand biomedical language. Fine-tuning adds a new classification head — a simple linear layer with 3 outputs — and trains the whole thing on 900 PubMedQA examples. The pre-trained weights give us a huge head start; we're nudging them for a specific task rather than learning language from scratch. This is why fine-tuning 900 examples takes 8 minutes on GPU while pre-training BiomedBERT took weeks on a cluster."

"How does the tokenizer work?"

Strong answer:

"The tokenizer converts text into integers the model can process. It uses sub-word tokenization — words are split into pieces so rare or medical terms can be represented from familiar sub-parts rather than marked as unknown. For text pair classification, I pass the question and abstract as two separate arguments. The tokenizer formats them as: [CLS] question [SEP] abstract [SEP] [PAD]... with three output arrays: input_ids for the token integers, attention_mask to tell the model which tokens are real vs padding, and token_type_ids to distinguish which tokens are the question versus the abstract. Crucially, the tokenization at inference must exactly match what was used during training — different padding or max_length would break the model."

"What is class imbalance and how did you handle it?"

Strong answer:

"Class imbalance is when some classes have far more training examples than others. In PubMedQA, 'yes' is 55% of examples, 'no' is 30%, and 'maybe' is only 15%. Without handling this, the model learns that predicting 'yes' is rewarded most often and stops predicting 'maybe' entirely — my first run confirmed this: 'maybe' F1 was 0.00. I fixed it with weighted cross-entropy loss. Using sklearn's compute_class_weight with 'balanced' strategy, I computed inverse-frequency weights: 'maybe' got about 2.2x weight vs 'yes' at 0.6x. I implemented this by subclassing HuggingFace's Trainer and overriding compute_loss to use nn.CrossEntropyLoss with these weights. The result was 'maybe' F1 jumping from 0.00 to 0.37 and macro F1 improving from 0.36 to 0.51."

"Why macro F1 and not accuracy?"

Strong answer:

"Accuracy is misleading with imbalanced classes. A model that always predicts 'yes' achieves 55% accuracy — better than random — while being completely useless for 'no' and 'maybe'. Macro F1 computes F1 separately for each class and averages equally, regardless of frequency. It cares just as much about performance on the 15% 'maybe' class as the 55% 'yes' class. For any real clinical decision support system, you absolutely need the model to handle all outcome categories — so macro F1 is the honest metric. I also used it as metric_for_best_model in TrainingArguments so the Trainer saves the checkpoint that maximizes macro F1, not just accuracy."

"What does the Trainer class do?"

Strong answer:

"Trainer is HuggingFace's high-level training loop. It handles everything: batching and shuffling data, running the forward pass, computing the loss, running backpropagation, stepping the AdamW optimizer, evaluating on the validation set after each epoch, saving checkpoints, logging metrics, and pushing to HuggingFace Hub. Without Trainer, I'd write all of this manually. In this project, I subclassed Trainer to override just one method — `compute_loss` — to implement weighted cross-entropy. All the rest of the training infrastructure stayed unchanged."

"Why did you use warmup steps?"

Strong answer:

"At the start of training, the classification head has completely random weights. If we immediately apply the full learning rate, we get very large gradient updates that can destroy the carefully pre-trained BiomedBERT weights. Warmup linearly increases the learning rate from 0 to the target ($2e-5$) over the first 34 steps. This gives the random classification head time to learn something reasonable before the full learning rate kicks in, protecting the valuable pre-trained representations."

"Your accuracy is only 57% — isn't that bad?"

Strong answer:

"It's modest, but honest and expected. The dataset has only 1000 expert-labeled examples, of which we use 900 for training. The 'maybe' class is genuinely ambiguous — even human annotators disagree on it, which is why the dataset authors marked those cases 'maybe' rather than forcing a yes/no answer. 57% is meaningfully above random (33%) and above the majority-class baseline (45% by always predicting 'yes'). The more important metric is macro F1 at 0.51, which shows the model is learning to handle all three classes. For a portfolio project, I think demonstrating honest evaluation and the ability to diagnose and fix real ML problems (class imbalance) is more valuable than chasing a high number on a small dataset."

"How does the Streamlit app load the model efficiently?"

Strong answer:

"I use `@st.cache_resource`. Streamlit reruns the entire Python script on every user interaction — every button click, every keystroke. Without caching, the 440MB model would be downloaded and loaded fresh on every click, which is obviously unusable. `@st.cache_resource` stores the return value of `load_model()` — the tokenizer and model objects — in memory after the first call. All subsequent reruns skip the function body entirely and return the cached objects instantly. I specifically use `cache_resource` rather than `cache_data` because PyTorch models can't be pickled reliably — `cache_resource` stores the object reference directly."

"What would you do to improve the results?"

Strong answer (pick 2-3):

1. **More data** — the `pqa_unlabeled` split has 61,000 examples. I could use semi-supervised learning: fine-tune on labeled data, generate pseudo-labels for unlabeled data, then train on both.
2. **Larger model** — BiomedBERT `large` (24 layers vs 12) would likely give better results at the cost of more compute.
3. **Hyperparameter tuning** — try different learning rates (1e-5, 3e-5, 5e-5) and warmup ratios using Optuna or HuggingFace's built-in HPO.
4. **Longer training with early stopping** — 5-6 epochs with `EarlyStoppingCallback` rather than a fixed 3 epochs.
5. **Data augmentation** — back-translation or synonym replacement to artificially increase the 'maybe' class.

Quick Facts to Know

Fact	Value
Base model	microsoft/BiomedNLP-BiomedBERT-base-uncased-abstract
Model parameters	~110 million
Pre-training data	21 million PubMed abstracts
Fine-tuning dataset	PubMedQA <code>pqa_labeled</code> , 1000 examples

Train/val split	900 / 100
Task	3-class text classification (yes/no/maybe)
Best metric	Macro F1 = 0.51
Training time	~8 min on T4 GPU, ~60 min on CPU
Max sequence length	512 tokens
Learning rate	2e-5
Batch size	16 (GPU)
Epochs	3
Key technique	Weighted cross-entropy loss for class imbalance